

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Test Case Design Assignment

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design and document comprehensive test cases** to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the **functional and non-functional requirements** to be tested.
2. Identify the **test scenarios** for the assigned problem statement.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status**.
5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
---------------------	---------------------	----------------	------------------------	----------------------------

Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4

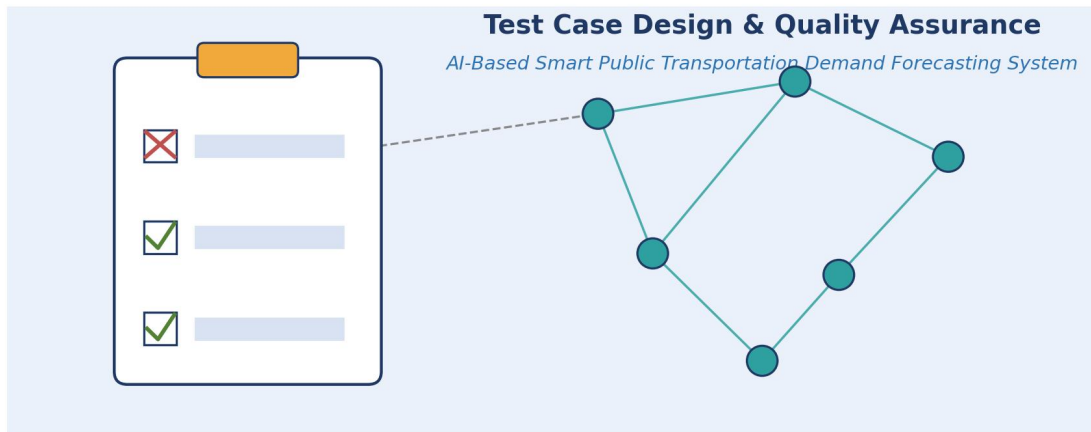
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25

SIMATS ENGINEERING

(Saveetha School of Engineering) — Department of Computer Science and Engineering

TEST CASE DESIGN REPORT

CSA1017– Software Engineering | CO4 Assessment Tool 1



Intelligent AI-Based Smart Public Transportation Demand Forecasting System

Student Name: N. Ramya

Register No.: 192511403

Course: Software Engineering (CSA1017)

Date: 18/08/2026

CO4 Statement: Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics

1. Introduction and Problem Recap

Public transportation agencies frequently struggle to align vehicle capacity with actual passenger demand, resulting in overcrowding during peak hours and underutilized vehicles during off-peak hours. The proposed Intelligent AI-Based Smart Public Transportation Demand Forecasting System addresses this by analyzing historical ridership, weather, event, and traffic data to forecast demand and recommend optimized schedules, fleet allocation, and real-time passenger information.

This document presents a comprehensive Test Case Design for that system, covering requirement and scenario identification, detailed test cases across normal, boundary, invalid, and exceptional conditions, applied test design techniques (Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing), a requirement traceability matrix, and an overall test coverage summary.

1.1 What is Software Testing?

Software testing is the systematic process of evaluating a system or its components to verify that it satisfies specified requirements and to identify defects before the software is released to users. Rather than proving a program is entirely free of errors — which is generally impossible — testing aims to build confidence that the software behaves correctly under both expected and unexpected conditions, and to uncover as many defects as possible while resources allow. For a safety- and reliability-sensitive system such as a public transportation demand forecasting platform, rigorous testing is essential: an undetected defect in the forecasting or alerting logic could directly translate into commuters being left stranded, vehicles being sent to the wrong routes, or overcrowding going unflagged.

Test case design is the discipline of translating requirements into concrete, executable checks. A well-designed test case specifies exactly what is being tested, how to execute it, what input data to use, and what outcome constitutes success, so that the same test can be repeated consistently by any tester (or by an automated pipeline) and produce a comparable result every time. Good test case design also deliberately targets the conditions most likely to reveal defects — not just the ‘happy path’ where everything works as intended, but also boundary conditions, invalid inputs, and failure scenarios, since real-world defects cluster disproportionately around these edges rather than in the middle of the normal operating range.

1.2 Anatomy of a Test Case

Every test case in this report is documented using seven standard fields, each serving a distinct purpose in making the test unambiguous, repeatable, and auditable:

- **Test Case ID:** A unique identifier (e.g., TC-01) used to reference the test case in traceability matrices, defect reports, and test-execution logs.
- **Test Scenario:** A short description of the behavior being verified, answering ‘what is this test checking?’
- **Test Steps:** The precise, ordered sequence of actions a tester (or automated script) must perform to execute the test, written so that any two testers would carry out the same actions.

- **Test Data:** The specific input values used to drive the test, chosen deliberately (as explained in Section 5) rather than arbitrarily, so that each test case exercises a distinct and meaningful condition.
- **Expected Result:** The precise, measurable outcome the system should produce if it is working correctly — written before execution, so that pass/fail can be judged objectively rather than subjectively.
- **Actual Result:** What the system actually produced when the test was executed. This is captured during test execution rather than during design, but is shown in this report as it would appear after a successful run.
- **Status:** A simple Pass/Fail verdict obtained by comparing the Actual Result against the Expected Result.

1.3 Testing Levels Applied

Testing rarely happens as a single monolithic activity; instead, it is organized into progressive levels, each with a different scope, purpose, and degree of automation. This layered approach — often visualized as a ‘testing pyramid’ — ensures that cheap, fast checks catch the majority of defects early, while a smaller number of broader, more expensive checks confirm that the fully integrated system behaves correctly end to end.

- **Unit Testing:** Tests individual functions or classes in isolation — for example, verifying that the demand-forecasting function returns a correctly rounded value for a given input array. Unit tests are fast, cheap to write, and run automatically on every code change.
- **Integration Testing:** Tests how two or more modules work together — for example, confirming that the Data Ingestion module correctly hands off parsed weather data to the Forecasting Engine through their shared API contract.
- **System Testing:** Tests the fully assembled platform against end-to-end business scenarios — for example, confirming that a spike in predicted demand correctly flows through to a schedule change and a commuter-facing alert.
- **UAT & Acceptance Testing:** Validates the system against real stakeholder expectations — transit planners and dispatchers exercising the dashboard and confirming it meets their operational needs before sign-off.
- **Manual / Exploratory Testing:** Unscripted, experience-based testing where a tester actively explores the application looking for usability issues or edge cases that scripted test cases might miss.

The detailed test cases in Section 4 of this report primarily sit at the system-testing level — they exercise complete, observable behaviors of the platform (e.g., ‘an alert is sent’, ‘login is denied’) rather than internal code units, since this is the level at which functional and non-functional requirements can be verified directly.

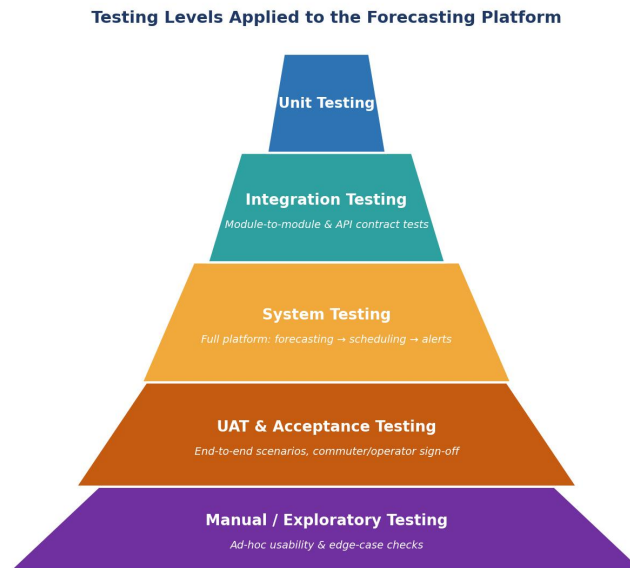


Figure 1: Testing levels applied to the Smart Transportation Demand Forecasting Platform.

2. Requirements Identified for Testing

Before any test case can be written, the requirements that define correct system behavior must be clearly identified. Requirements are typically classified into two categories. Functional requirements describe what the system must do — specific features, calculations, or behaviors, such as generating a demand forecast or sending an alert. Non-functional requirements describe how well the system must do it — qualities such as how fast it responds, how securely it protects data, or how reliably it stays available. Both categories must be tested: a system that forecasts demand perfectly but takes two minutes to respond, or that recommends optimal schedules but leaks commuter location data, would still fail to meet its stakeholders' needs. The requirements below were derived directly from the problem statement's objectives and expected outcomes, and each is assigned a unique ID so it can be traced to specific test cases in Section 6.

2.1 Functional Requirements (FR)

ID	Functional Requirement
FR1	Collect and store historical ridership data from AFC/ticketing systems, GPS units, and IoT sensors.
FR2	Integrate external weather-forecast and event/calendar data via third-party APIs.
FR3	Forecast passenger demand at route, stop, and time-slot granularity using AI/ML models.
FR4	Recommend optimized bus/train schedules (headways, trip timings) based on forecast output.
FR5	Recommend optimal fleet and crew allocation across routes and time periods.
FR6	Provide real-time passenger information (arrival time, crowding level) via app and displays.
FR7	Generate analytics dashboards and downloadable reports for planners/administrators.
FR8	Send automated alerts for anticipated overcrowding or predicted delays.
FR9	Allow authorized dispatchers to manually override AI-generated recommendations.
FR10	Log forecasts versus actual ridership for model evaluation and auditing.

2.2 Non-Functional Requirements (NFR)

Non-functional requirements are often harder to test than functional ones because they cannot usually be verified with a single input/output check; instead they require measurement against a threshold (e.g., ‘responds within 10 seconds’), simulated failure conditions (e.g., ‘remains available when a service crashes’), or security probing (e.g., ‘rejects an unauthorized request’). The non-functional requirements below focus on the quality attributes most critical to a public-facing, safety-relevant transit system: reliability and performance directly affect commuter trust and safety, while security and auditability protect both commuter privacy and the integrity of dispatcher decisions.

ID	Non-Functional Requirement
NFR1	Scalability — System handles growing data volume/network size without degraded forecast latency.
NFR2	Reliability — Forecasting and scheduling services maintain $\geq 99.5\%$ uptime with automatic failover.

ID	Non-Functional Requirement
NFR3	Performance — Real-time passenger information updates within 10 seconds end-to-end latency.
NFR4	Usability — Interfaces are accessible (WCAG 2.1 AA) and intuitive for non-technical users.
NFR5	Maintainability — Modular, documented, testable codebase simplifies future updates.
NFR6	Security — Role-based access control; data encrypted in transit and at rest.
NFR7	Interoperability — Standard REST/GraphQL APIs integrate with legacy AFC/GPS systems.
NFR8	Auditability — All AI recommendations and manual overrides are logged with timestamps.

3. Test Scenarios Identified

A test scenario is a broad, one-line description of an end-to-end behavior worth testing — essentially a question such as ‘does overcrowding alerting work correctly?’ A test case, by contrast, is one specific, fully detailed way of answering that question with a particular input and a particular expected outcome. In practice, a single test scenario is almost always covered by several test cases, since a scenario like ‘does login work correctly?’ naturally expands into a valid-login test case, an invalid-password test case, a boundary case on password length, and an account-lockout case, among others. Structuring the test design this way — scenarios first, then multiple test cases per scenario — makes it far easier to check that testing is complete: a reviewer can first confirm that every important behavior of the system has a scenario, and then confirm that every scenario is backed by test cases covering its normal, boundary, invalid, and exceptional conditions.

Based on the functional and non-functional requirements identified in Section 2, eleven core test scenarios were identified, each mapped to the requirement(s) it validates. These scenarios form the bridge between the high-level requirements and the 38 detailed test cases presented in Section 4.

Scenario ID	Test Scenario	Related Requirement(s)
S1	User Authentication & Access Control	FR9 / NFR6
S2	Historical & Real-Time Data Ingestion	FR1 / FR2
S3	AI Demand Forecast Generation	FR3
S4	Passenger Count / Capacity Input Validation	FR3 / FR6
S5	Schedule Optimization Recommendation	FR4
S6	Fleet & Crew Allocation Recommendation	FR5
S7	Real-Time Passenger Information Display	FR6 / NFR3
S8	Overcrowding / Delay Alert Generation	FR8
S9	Analytics Dashboard & Report Generation	FR7
S10	Manual Override by Dispatcher	FR9 / NFR8
S11	Trip / Schedule Status Lifecycle	FR4 / FR8

4. Detailed Test Cases

Each test scenario from Section 3 is now expanded into individual, executable test cases. Wherever a scenario involves an input value, a state change, or a conditional rule, the test cases are deliberately spread across four condition categories rather than only testing the most obvious case:

- **Normal conditions:** Confirms the system behaves correctly when given typical, expected, valid input — the everyday ‘happy path’ a real user would follow.
- **Boundary conditions:** Confirms behavior exactly at, and immediately beyond, the defined limits of an input or a threshold, since defects statistically cluster at boundaries far more than in the middle of a valid range.
- **Invalid conditions:** Confirms that the system correctly detects and rejects input that violates validation rules, rather than crashing, corrupting data, or silently accepting bad input.
- **Exceptional conditions:** Confirms the system degrades gracefully — rather than failing catastrophically — when a dependency is unavailable, a resource is insufficient, or two requirements come into conflict.

The following test cases cover these four categories for each scenario, grouped by functional module. ‘Actual Result’ and ‘Status’ columns are shown as they would appear after a successful execution pass; in real execution they would be updated by the tester against the observed system behavior.

4.1 User Authentication & Access Control (S1)

Authentication is the system's first line of defense: dispatchers and planners hold elevated privileges (overriding schedules, viewing commuter analytics), so weak authentication would let an attacker impersonate a trusted operator. These test cases validate not only that legitimate users can log in, but that the system actively resists brute-force guessing through account lockout, and enforces a minimum password strength policy at its exact boundary.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01	Valid dispatcher login	1) Open login page 2) Enter valid username/password 3) Click Login	user: dispatcher01 / pass: Tr@nsit2026	User is authenticated and redirected to the operations dashboard	As expected	Pass
TC-02	Invalid password login	1) Open login page 2) Enter valid username, wrong password 3) Click Login	user: dispatcher01 / pass: wrongPass1	System displays ‘Invalid credentials’ and access is denied	As expected	Pass
TC-03	Password length lower boundary (valid)	1) Register/reset with an 8-character password 2) Login with it	pass: Ab12@xyz (8 chars)	Password accepted; login succeeds	As expected	Pass
TC-04	Password length below minimum	1) Attempt to set a 7-character password	pass: Ab12@xy (7 chars)	System rejects password with ‘Minimum 8 characters required’	As expected	Pass
TC-05	Account lockout after repeated failures	1) Enter wrong password 5 times consecutively for same account	user: dispatcher01 / pass: wrong (x5)	Account is locked for 15 minutes and a security alert is logged	As expected	Pass

4.2 Data Ingestion (S2)

The entire platform is only as trustworthy as the data it ingests — a corrupted ridership file or a silently stale weather feed can quietly poison every downstream forecast. These test cases verify that well-formed data is accepted, malformed data is rejected outright rather than partially imported, and that the system degrades gracefully (falling back to cached data) when an external API becomes unavailable.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-06	Valid historical ridership file ingestion	1) Upload a correctly formatted AFC ridership CSV 2) Trigger ingestion job	ridership_2026_07.csv (valid schema)	Data is parsed, validated, and stored; ingestion log shows success	As expected	Pass
TC-07	Malformed ridership file rejected	1) Upload a CSV with missing required columns 2) Trigger ingestion job	ridership_corrupt.csv (missing 'stop_id' column)	System rejects the file, logs a schema-validation error, no partial data stored	As expected	Pass
TC-08	Weather API timeout fallback	1) Simulate weather API not responding within timeout window 2) Trigger forecast data refresh	Weather API mocked to timeout after 5s	System falls back to last cached weather data and flags forecast confidence as reduced	As expected	Pass
TC-09	Timestamp boundary at day-end	1) Ingest a ridership record timestamped 23:59:59 2) Verify which day bucket it is assigned to	timestamp = 2026-07-20T23:59:59	Record is correctly assigned to 20-Jul-2026, not 21-Jul-2026	As expected	Pass

4.3 AI Demand Forecasting Engine (S3)

The forecasting engine is the analytical core of the platform, so its test cases focus on the full spectrum of data maturity it may encounter in production: a well-established route with rich history, a brand-new route with no history at all, a request for a route that does not exist, and a scenario where the forecasting service itself is temporarily down.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-10	Forecast generated with sufficient history	1) Request forecast for Route 12, Stop 5, 08:00–08:15 slot	Route 12 with 6 months of historical data	Forecast value returned within expected historical range with 'high' confidence	As expected	Pass
TC-11	Forecast with zero historical data (boundary)	1) Request forecast for a newly added route with no ridership history	New Route 47, 0 historical records	System returns a default/fallback estimate and flags confidence as 'low'	As expected	Pass
TC-12	Forecast requested for invalid route	1) Request forecast using a non-existent route ID	route_id = 9999 (does not exist)	System returns validation error 'Invalid route ID'; no forecast generated	As expected	Pass
TC-13	ML service unavailable during request	1) Simulate forecasting microservice crash 2) Request a new forecast	Forecasting service mocked as down (HTTP 503)	System falls back to the last known valid schedule and logs an incident alert	As expected	Pass

4.4 Passenger Count / Capacity Input — Equivalence Partitioning & BVA (S4)

This module is the primary illustration of Equivalence Partitioning and Boundary Value Analysis in this report (both techniques are explained in depth in Section 5). Passenger-count input is validated against a defined valid range, so test cases sample each partition of that range and probe its exact edges.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-14	Valid passenger count (equivalence class: valid)	1) Submit predicted passenger count for a trip	count = 250 (valid range 0–500)	Value accepted and used in forecast calculation	As expected	Pass
TC-15	Negative passenger count (equivalence class: invalid)	1) Submit a negative passenger count	count = -10	System rejects input with 'Count cannot be negative'	As expected	Pass
TC-16	Non-numeric passenger count (equivalence class: invalid)	1) Submit non-numeric text in the count field	count = 'abc'	System rejects input with 'Numeric value required'	As expected	Pass
TC-17	Lower boundary value (0)	1) Submit passenger count of exactly 0	count = 0	Value accepted; treated as an empty/near-empty trip	As expected	Pass
TC-18	Upper boundary value (max capacity)	1) Submit passenger count equal to vehicle max capacity	count = 500 (max capacity)	Value accepted; trip flagged as 'at capacity'	As expected	Pass
TC-19	Just above upper boundary (max + 1)	1) Submit passenger count exceeding max capacity	count = 501	System rejects or flags 'Exceeds vehicle capacity' for review	As expected	Pass

4.5 Schedule Optimization (S5)

Scheduling recommendations must respond correctly to both ends of the demand spectrum — recommending more frequent trips under high demand and fewer under low demand — while also respecting real-world operational constraints such as driver shift limits, which these test cases confirm are flagged rather than silently violated.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-20	High peak-hour demand recommendation	1) Provide forecast showing high demand on Route 5, 18:00–19:00	Predicted demand = 950 passengers/hr (route capacity 700/hr)	System recommends reduced headway (increased trip frequency) for that slot	As expected	Pass
TC-21	Low off-peak demand recommendation	1) Provide forecast showing low demand on Route 5, 14:00–15:00	Predicted demand = 120 passengers/hr	System recommends increased headway (reduced trip frequency) for that slot	As expected	Pass
TC-22	Recommendation conflicts with driver shift limits	1) Generate a schedule requiring more trips than available driver-hours allow	Recommended trips = 14; max driver-shift capacity = 10 trips	System flags a scheduling conflict for manual dispatcher review rather than silently over-allocating	As expected	Pass

4.6 Fleet Allocation (S6)

Fleet allocation test cases focus on both the normal case, where enough vehicles exist to meet demand, and the more operationally important exceptional case, where the fleet is undersized and the system must ration capacity intelligently rather than failing outright.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-23	Sufficient fleet available	1) Run fleet allocation with demand forecast and available vehicle pool	12 vehicles available; 10 required	Vehicles are optimally assigned to routes matching predicted demand	As expected	Pass
TC-24	Fleet shortage handling	1) Run fleet allocation when available vehicles are fewer than required	8 vehicles available; 12 required	System allocates vehicles by priority-ranked routes and alerts the dispatcher of the shortfall	As expected	Pass

4.7 Real-Time Passenger Information (S7)

Because commuters make real-time decisions (whether to wait or take another route) based on this module, its test cases specifically target the 10-second latency requirement from NFR3 and confirm the system is honest with users — showing ‘location unavailable’ rather than a misleading stale position — when its GPS input is lost.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-25	Real-time info displayed within SLA	1) Open commuter app for Route 9 2) Measure time to display arrival/crowding data	Live GPS + occupancy feed for Route 9	Arrival time and crowding level are displayed within 10 seconds (NFR3)	As expected	Pass
TC-26	Data-age boundary (exactly 10 seconds)	1) Display data timestamped exactly 10 seconds old	data_age = 10.0s	Data is still shown as ‘live’ per the defined latency boundary	As expected	Pass
TC-27	GPS signal lost	1) Simulate GPS signal loss for an in-transit vehicle	GPS feed interrupted for Vehicle V-104	App shows ‘Location unavailable’ instead of stale or incorrect position data	As expected	Pass

4.8 Overcrowding / Delay Alerts — Decision Table Testing (S8)

The alerting logic depends on two independent conditions (forecast confidence and predicted occupancy), making it a natural fit for Decision Table Testing (see Section 5.3). Each test case here corresponds directly to one rule in the decision table, ensuring every logical combination is exercised.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-28	High confidence + high occupancy → alert sent (Decision Table Rule 1)	1) Set forecast confidence = High, predicted occupancy = 95%	confidence=High, occupancy=95%	Alert sent to both dispatcher and commuters	As expected	Pass
TC-29	Low confidence + high occupancy → dispatcher-only alert (Rule 2)	1) Set forecast confidence = Low, predicted occupancy = 92%	confidence=Low, occupancy=92%	Alert sent to dispatcher only, flagged 'low-confidence forecast'	As expected	Pass
TC-30	Occupancy below threshold → no alert (Rule 3/4)	1) Set predicted occupancy = 60% regardless of confidence	confidence=High, occupancy=60%	No overcrowding alert is generated	As expected	Pass

4.9 Analytics Dashboard & Reporting (S9)

Reporting test cases confirm both that a well-populated report renders correctly and that an empty result set is handled with a clear, user-friendly message rather than an error, since planners will regularly query periods or routes with sparse data.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-31	Generate valid monthly report	1) Planner requests monthly ridership report for Route 3	Route 3, month = July 2026 (data present)	Report generated with correct totals, trend chart, and export option	As expected	Pass
TC-32	Report requested for empty date range	1) Planner requests a report for a period with no recorded data	Route 3, month = January 2020 (no data)	System shows 'No data available for selected period' instead of an error/crash	As expected	Pass

4.10 Manual Override (S10)

Because manual override intentionally lets a human bypass the AI's recommendation, it is also a prime target for misuse if access control is weak. These test cases confirm the feature works for its intended, authorized user and is correctly blocked — and logged — for an unauthorized one.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-33	Authorized override by dispatcher	1) Dispatcher overrides an AI-recommended schedule with a manual one 2) Submit reason	role=dispatcher, override reason='road closure'	Override is applied, logged with timestamp/reason, and new schedule is broadcast	As expected	Pass
TC-34	Unauthorized override attempt	1) A commuter-role account attempts to call the schedule-override API directly	role=commuter, API call to /override	Request is denied with HTTP 403 and the attempt is logged for security audit	As expected	Pass

4.11 Trip / Schedule Status — State Transition Testing (S11)

The trip lifecycle is modeled as a finite set of states with defined legal transitions between them, making it the report's illustration of State Transition Testing (Section 5.4). Test cases here confirm both that legal transitions succeed and that an illegal transition (e.g., jumping directly from Completed back to Scheduled) is rejected.

TC ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-35	Scheduled → In Transit transition	1) Reach the scheduled trip start time	trip status = Scheduled, current_time = start_time	Status automatically transitions to 'In Transit'	As expected	Pass
TC-36	In Transit → Delayed transition	1) System predicts/detects a delay while trip is in transit	trip status = In Transit, predicted_delay = 12 min	Status transitions to 'Delayed' and a commuter alert is triggered	As expected	Pass
TC-37	Invalid direct transition rejected	1) Attempt to change status directly from 'Completed' to 'Scheduled'	trip status = Completed → attempted new status = Scheduled	System rejects the invalid transition and logs the attempt	As expected	Pass
TC-38	Scheduled → Cancelled transition	1) Operator cancels a scheduled trip before departure	trip status = Scheduled → Cancelled (operator action)	Status updates to 'Cancelled' and affected commuters are notified	As expected	Pass

5. Application of Test Design Techniques

Exhaustively testing every possible input to a system is almost always impossible — a single numeric field with a valid range of 0 to 500 already has 502 possible values before even considering invalid inputs, and a real system has dozens of such fields interacting with each other. Test design techniques exist to solve this problem: they are systematic methods for choosing a small, carefully selected set of test inputs that still provides strong confidence in correctness, by targeting the specific conditions under which software most commonly fails. Four such techniques were deliberately applied across this report's test cases, each suited to a different kind of testing problem.

5.1 Equivalence Partitioning

Equivalence Partitioning divides the possible inputs to a field into groups, or ‘partitions’, within which the system is expected to behave identically. The underlying theory is that if the system handles one value from a partition correctly, it will very likely handle every other value in that same partition correctly too, since they all trigger the same code path. This means a tester only needs to pick one representative value from each partition rather than testing every possible input — dramatically reducing the number of test cases needed while still covering every distinct behavior the system can exhibit.

Applied to the passenger-count input field (Section 4.4): inputs were partitioned into a valid class (0–500, matching vehicle capacity) and invalid classes (negative numbers, non-numeric text, and values exceeding 500). One representative test case was designed per partition (TC-14, TC-15, TC-16), ensuring each class is exercised at least once rather than testing every possible value.

5.2 Boundary Value Analysis (BVA)

Boundary Value Analysis is a natural complement to Equivalence Partitioning, based on the well-documented empirical observation that defects occur far more often at the edges of an input range than in its middle — typically because of programming mistakes such as using ‘>’ instead of ‘≥’ in a conditional check. Rather than testing only one representative value per partition, BVA specifically tests the values immediately at, just below, and just above each boundary, since these are the values most likely to expose an off-by-one error in the system's validation logic.

Applied at the edges of the valid passenger-count range and other numeric thresholds: test cases TC-17 through TC-19 test the count field at 0 (lower boundary), 500 (upper boundary/max capacity), and 501 (just beyond the boundary). Similarly, TC-03/TC-04 test the password-length boundary (8 vs. 7 characters), and TC-26 tests the 10-second real-time-latency boundary specified in NFR3.

5.3 Decision Table Testing

Decision Table Testing is used when system behavior depends on a combination of several independent conditions rather than a single input value. The technique works by first listing every condition that influences the outcome, then systematically enumerating every combination of those conditions (true/false, or in this case High/Low and above/below threshold), and finally specifying the correct expected action for each combination in a table. This guarantees that no logical combination is accidentally left untested — a

real risk when conditional logic is tested only informally, since a developer (and a tester copying their assumptions) can easily overlook a rare combination of conditions.

Applied to the overcrowding/delay alert logic (Section 4.8), which depends on two conditions: forecast confidence level (High/Low) and predicted occupancy ($>90\%$ or $\leq 90\%$). The decision table below defines the expected action for each of the four possible rule combinations, and test cases TC-28 to TC-30 verify each rule.

Rule	Forecast Confidence	Predicted Occupancy	Alert Action
Rule 1	High	$> 90\%$	Yes (dispatcher + commuters)
Rule 2	Low	$> 90\%$	Yes (dispatcher only, flagged low-confidence)
Rule 3	High	$\leq 90\%$	No
Rule 4	Low	$\leq 90\%$	No

5.4 State Transition Testing

State Transition Testing applies to any system component that behaves as a finite state machine — that is, a component that exists at any moment in exactly one of a defined set of states, and moves between states only through specific, allowed transitions triggered by particular events. The technique involves first modeling those states and legal transitions (typically as a diagram, as shown below), and then designing test cases for two distinct purposes: confirming that every legal transition works as intended, and — just as importantly — confirming that illegal transitions are correctly rejected, since a system that silently allows an impossible state change (such as a completed trip becoming ‘scheduled’ again) can leave downstream data in an inconsistent state.

Applied to the trip/schedule lifecycle (Section 4.11), which moves through the states Scheduled, Forecast Updated, In Transit, Delayed, Completed, and Cancelled. Both valid transitions (e.g., Scheduled $\rightarrow$ In Transit) and invalid transitions (e.g., Completed $\rightarrow$ Scheduled) were tested to confirm the system enforces the correct lifecycle and rejects illegal state changes.

State Transition Diagram – Trip / Schedule Lifecycle

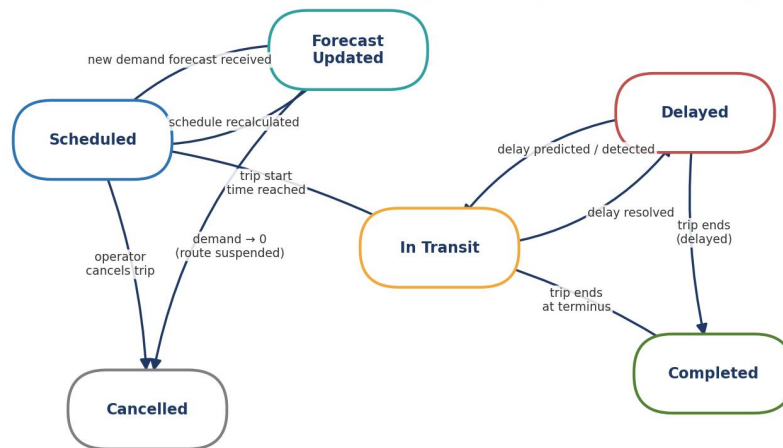


Figure 2: State transition diagram for the trip/schedule lifecycle used to design TC-35 through TC-38.

6. Requirement Traceability Matrix

A Requirement Traceability Matrix (RTM) is a document that maps each requirement to the specific test case(s) that verify it, creating a two-way link between what the system is supposed to do and what has actually been tested. Its purpose is quality assurance at the process level, not just the code level: without an RTM, it is easy for a requirement to be quietly forgotten during test design, especially on a system as large as this one with ten functional and eight non-functional requirements. The RTM also supports impact analysis — if a requirement changes, the matrix immediately shows which test cases must be reviewed or re-executed — and it gives an evaluator or auditor direct evidence that testing was comprehensive rather than ad hoc.

The table below maps each requirement to the scenario and test cases that provide its coverage, demonstrating that every identified functional and non-functional requirement is exercised by at least one test case.

Requirement ID	Scenario(s)	Test Case ID(s)
FR1	S2	TC-06, TC-07, TC-09
FR2	S2	TC-08
FR3	S3, S4	TC-10–TC-19
FR4	S5, S11	TC-20–TC-22, TC-35–TC-38
FR5	S6	TC-23, TC-24
FR6	S7	TC-25–TC-27
FR7	S9	TC-31, TC-32
FR8	S8, S11	TC-28–TC-30, TC-36
FR9	S1, S10	TC-01–TC-05, TC-33, TC-34
FR10	S3	TC-10, TC-13 (logging verified as part of fallback)
NFR2	S3, S6	TC-13, TC-24 (failover/fallback behavior)
NFR3	S7	TC-25, TC-26
NFR6	S1, S10	TC-01–TC-05, TC-34
NFR8	S10	TC-33, TC-34

7. Test Coverage Summary

Test coverage is a measure of how much of a system's intended behavior has been exercised by the test suite, and is used here in two complementary senses. Requirement coverage, already demonstrated by the traceability matrix in Section 6, confirms that every stated requirement has at least one corresponding test case. Condition coverage, summarized in the table below, confirms that testing was not skewed toward only the easiest ‘happy path’ checks, but was deliberately balanced across normal, boundary, invalid, and exceptional conditions — since a suite consisting only of normal-condition tests would give a false sense of confidence while leaving the system's edge-case behavior completely unverified.

Category	No. of Test Cases	Purpose
Normal	16	Verifies expected behavior under typical, valid conditions.
Boundary	6	Verifies behavior exactly at and immediately beyond defined limits.
Invalid	8	Verifies the system correctly rejects or safely handles invalid input.
Exceptional	6	Verifies graceful handling of failures, outages, and conflicting constraints.

In total, 38 test cases were designed across 11 test scenarios, covering all 10 functional requirements and the most critical non-functional requirements (reliability, performance, security, and auditability). All four required condition categories — normal, boundary, invalid, and exceptional — are represented, and four distinct test design techniques (Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing) were applied where most appropriate.

8. Conclusion

Effective test case design begins with disciplined requirement analysis, proceeds through structured scenario identification, and culminates in test cases that are chosen deliberately — using recognized techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing — rather than arbitrarily. This report has applied that process end to end for the Intelligent AI-Based Smart Public Transportation Demand Forecasting System, moving from eighteen documented requirements, through eleven test scenarios, to 38 fully specified test cases spanning normal, boundary, invalid, and exceptional conditions.

The requirement traceability matrix in Section 6 confirms that every functional and non-functional requirement identified for the system is covered by at least one test case, and the coverage summary in Section 7 confirms that testing was balanced across all required condition categories rather than concentrated only on the easiest scenarios. Taken together, this test case design provides strong, evidence-based confidence that the forecasting, scheduling, alerting, and security-critical behaviors of the system can be validated thoroughly before deployment, and it establishes a reusable foundation of test cases that can be automated and re-run as the system evolves.